



Review article

Use of peer code review to identify potential security vulnerabilities in open source software: A Systematic Literature Review

Md Niaz Morshed, Jeffrey C. Carver^{ID*}

Department of Computer Science, The University of Alabama, 35487, Tuscaloosa, USA

ARTICLE INFO

Keywords:

Code review

Security vulnerabilities

Systematic Literature Review

ABSTRACT

Context: Security is a major software development concern. Security vulnerabilities in Open-Source Software (OSS) can have severe consequences. OSS project participants widely employ peer code review to increase code quality and prevent security flaws. However, even with mandatory code reviews, many OSS projects still contain security flaws. The presence of these flaws highlights the importance of evaluating the effectiveness of code reviews in addressing security issues.

Objectives: This Systematic Literature Review analyzes how peer code review helps identify OSS security vulnerabilities. The study synthesizes existing evidence on vulnerability types identified or missed, current security practices, tool supports, reviewer challenges, and factors enhancing detection.

Method: We conducted a search across five major digital libraries. We screened 1067 studies and selected 20 primary studies using predefined inclusion and exclusion criteria. We analyzed the data using qualitative analysis and categorized vulnerabilities using the Common Weakness Enumeration (CWE) framework.

Results: Peer code reviews effectively detect low-hanging, localized, well-known vulnerabilities (e.g., buffer overruns, race conditions, crashes, XSS) but tend to miss subtle, complex, context-dependent issues (e.g., input validation weaknesses, access control weaknesses, resource lifecycle weaknesses). Effective security practices are tied to reviewer expertise, familiarity with the codebase, frequency of small code changes, and organizational support. Security analysis tools, particularly Static Application Security Testing (SAST) tools, reduce reviewer workload and improve detection, but cannot replace human judgment. Reviewers still suffer from constraints of security knowledge and experience, time, and comprehending complex systems. Reviewer effectiveness improves with security training, domain experience, contextual familiarity, formalized review processes, and favorable review environments.

Conclusion: Peer code review contributes to OSS security, but is not sufficient alone. Strengthening the effectiveness of peer code review requires integrating structured processes and checklists, security training, organizational incentives, and lightweight automated tool support to help reviewers detect subtle, complex vulnerabilities, and reinforcing secure development practices.

Contents

1. Introduction	2
2. Background	3
2.1. Software vulnerabilities	3
2.2. Peer code review	3
2.3. Peer code review practices and security	3
3. Methodology	3
3.1. Defining the research questions	4
3.2. Data source selection	4
3.3. Search string	4
3.4. Selection criteria	4
3.5. Primary study selection	4
3.6. Data extraction	5

* Corresponding author.

E-mail addresses: mmorshed@crimson.ua.edu (M.N. Morshed), carver@cs.ua.edu (J.C. Carver).

3.7.	Data synthesis	5
4.	Results	5
4.1.	Classification of security issues	5
4.2.	RQ1 What types of potential security vulnerabilities can be identified or missed during peer code review in OSS?	6
4.2.1.	High-level vulnerability types	6
4.2.2.	Identified vulnerabilities	6
4.2.3.	Missed vulnerabilities	7
4.3.	RQ2 What are the current security practices during peer code review in OSS?	7
4.3.1.	Reviewer involvement	7
4.3.2.	Reviewer expertise and code familiarity	7
4.3.3.	Code change characteristics	7
4.3.4.	Addressing vulnerabilities	7
4.3.5.	Human and organizational factors	8
4.4.	RQ3 How do security analysis tools contribute to improving OSS security during peer code reviews?	8
4.5.	RQ4 What challenges and limitations do reviewers face in ensuring security during peer code reviews in OSS?	8
4.5.1.	Insufficient security knowledge and expertise	8
4.5.2.	Time constraints and review fatigue	8
4.5.3.	Contextual and practical constraints	8
4.5.4.	Human factors and process limitations	9
4.6.	RQ5 What factors enhance a reviewer's ability to detect potential security vulnerabilities during peer code reviews in OSS?	9
4.6.1.	Security knowledge and expertise	9
4.6.2.	Review background and codebase familiarity	9
4.6.3.	Mental attitude and security awareness	9
4.6.4.	Training, support, and review environment	9
4.6.5.	Review duration and engagement	9
5.	Discussion	9
5.1.	RQ1 What types of potential security vulnerabilities can be identified or missed during peer code review in OSS?	9
5.2.	RQ2 What are the current security practices during peer code review in OSS?	9
5.3.	RQ3 How do security analysis tools contribute to improving OSS security during peer code reviews?	10
5.4.	RQ4 What challenges and limitations do reviewers face in ensuring security during peer code reviews in OSS?	10
5.5.	RQ5 What factors enhance a reviewer's ability to detect potential security vulnerabilities during peer code reviews in OSS?	10
5.6.	Emerging trends: AI-augmented code review for security in OSS	10
5.7.	Implications of OSS development context	11
6.	Threats to validity	11
6.1.	Search strategy and study selection	11
6.2.	Security vulnerability classification	11
6.3.	Scope and generalizability	11
6.4.	Reliability	11
7.	Conclusion	11
	CRedit authorship contribution statement	12
	Declaration of competing interest	12
	Appendix	12
	Data availability	12
	References	14

1. Introduction

Security vulnerabilities are a significant concern for software systems. Their exploitation can result in significant financial losses, privacy violations, and service disruptions. The impact of these vulnerabilities becomes even more pronounced if discovered after software deployment. Notable examples, including the Equifax [1] breach, which exposed sensitive personal information, and the Heartbleed [2] vulnerability, which enabled attackers to read sensitive memory, highlight the real-world impact of security vulnerabilities in widely used Open Source Software (OSS). The true prevalence of vulnerabilities is likely higher, as many remain undetected. In 2024, NIST reported over 39,000 vulnerabilities [3], underscoring the need for early detection and mitigation.

OSS has become a cornerstone of modern software development, fostering innovation and promoting collaboration across global communities. OSS development relies heavily on distributed, volunteer-driven collaboration, in which contributors have varying levels of expertise. This diversity of contributor skill level can lead to inconsistent code quality and increased vulnerabilities. Unlike proprietary projects with dedicated security teams, OSS relies on community members to identify and address security vulnerabilities. These vulnerabilities can

arise during development and remain undetected, posing significant risks to users and organizations. Due to the prevalence of OSS, these vulnerabilities can have wide-ranging effects.

Developers commonly use automated program analysis tools to support vulnerability detection [4]. These tools employ static techniques to identify insecure code patterns and dynamic techniques to expose runtime failures. However, prior studies have shown that such tools suffer from limitations, including high false-positive rates in static analysis and scalability challenges in dynamic analysis [5,6]. These limitations restrict their effectiveness when used in isolation.

Peer code review (a.k.a code review) is a widely adopted collaborative practice in which developers manually inspect code changes to improve quality and detect defects, including security vulnerabilities [7,8]. Unlike automated tools, peer review incorporates human judgment and contextual understanding, which are essential for identifying subtle or context-dependent security issues. Previous research indicates that code reviews are crucial for identifying and resolving security issues early on and contribute positively to the development of secure software [9–12]. An effective code review can reveal approximately 50% of the security issues present [13]. Consequently, many OSS and industrial projects mandate code review as part of their development workflow [14]. Despite this widespread adoption, security vulnerabilities continue to escape detection during mandatory reviews and surface after release [15].

Existing research on code review has largely focused on general software quality, bug detection, and adherence to coding standards [16–18]. Security-specific outcomes have received comparatively less attention, even though security vulnerabilities differ from general defects in their causes, impact, and detection requirements [19–21]. Although several empirical studies have demonstrated the potential of code reviews to improve security [22–24], it remains unclear how effective peer code reviews are in identifying security issues in OSS.

To address this gap, this paper presents a systematic literature review of code review for identifying potential security vulnerabilities in open-source software. The review synthesizes empirical evidence on the types of potential vulnerabilities detected and missed during reviews, existing security-oriented review practices, the role of automated security tools, challenges faced by reviewers, and factors that enhance the detection of potential vulnerabilities. By consolidating findings across studies, this work aims to clarify the current state of knowledge and identify limitations that hinder effective security-focused code reviews in OSS.

The key contributions of this study are as follows:

- A structured categorization of potential security vulnerability types that are commonly identified or missed during code reviews in OSS.
- A synthesis of current security practices applied during code reviews in OSS.
- An assessment of how security analysis tools support potential security vulnerability detection during code review.
- An identification of key challenges and limitations that hinder reviewers from effectively detecting potential security vulnerabilities.
- An analysis of factors that enhance reviewers' ability to detect potential security vulnerabilities.

The research artifacts associated with this systematic literature review, including the data extraction forms and the raw data of identified and missed potential vulnerabilities are provided as supplementary material via an online repository¹ to ensure transparency and reproducibility.

The paper is organized as follows: Section 2 provides background information. Section 3 outlines the research methodology used for our systematic literature review. Section 4 analyzes the results obtained. Discussion derived from our study is presented in Section 5. Threats to validity are examined in Section 6. Finally, Section 7 concludes the paper.

2. Background

This section provides the necessary background for the study by examining key topics in software security and peer code-review practices in OSS.

2.1. Software vulnerabilities

A software vulnerability is a flaw or weakness in software that attackers can exploit to compromise the confidentiality, integrity, or availability of a system [25]. Such vulnerabilities can lead to serious consequences, including data breaches, financial loss, and reputational damage [26]. To reduce these risks, organizations increasingly adopt a “shift-left” approach, which emphasizes detecting security issues earlier in the development lifecycle, including during code review [27].

Numerous security vulnerabilities can arise at various stages, including implementation, design, and system architecture. To support consistent analysis and communication, the security community categorizes vulnerabilities using the MITRE Common Weakness Enumeration

(CWE) framework [28]. CWE provides a standardized taxonomy in which each weakness is assigned a unique identifier and description, enabling systematic classification and comparison across studies. Software developers and researchers discuss security issues at different levels of abstraction. A *weakness* refers to a condition in software that may contribute to vulnerabilities under certain circumstances, whereas a *vulnerability* refers to a concrete and exploitable security flaw in a deployed system. Security databases such as Common Vulnerabilities and Exposures (CVE) typically catalog vulnerabilities after deployment, whereas developers and reviewers can identify weaknesses earlier during development activities such as peer code review. This distinction is particularly important in peer code review because reviewers examine insecure coding patterns and potential Vulnerabilities before deployment. In this review, we adopt the CWE framework to categorize the potential security vulnerabilities reported in the primary studies.

2.2. Peer code review

Peer code review is a collaborative practice in which developers other than the author examine code changes before they are integrated into the codebase. This process is typically iterative and continues until reviewers approve the code change for integration. Reviewers aim to detect defects, enforce coding standards, and improve software quality [29]. The practice originated from a systematic code review method known as code inspections [30]. This approach followed a structured and synchronous process where trained teams reviewed code through formal meetings.

Modern code review (MCR), introduced by Cohen [31] and later studied by Bacchelli and Bird [29], is a widely adopted form of code review. MCR is typically informal, tool-based, asynchronous, and focused on reviewing incremental code changes rather than entire codebases [29]. Tool support allows reviewers to inspect code diffs, leave comments, and iterate on changes efficiently.

2.3. Peer code review practices and security

Beyond general quality assurance, peer code review also plays an important role in software security. Prior studies show that code reviews can help identify certain classes of vulnerabilities and support knowledge sharing among developers [29,32,33]. In addition, McGraw emphasized that peer code review is crucial for identifying and fixing security vulnerabilities [9]. Howard [34] further highlighted that effective security-focused code review requires reviewers to understand common vulnerability patterns and prioritize high-risk code areas, such as legacy code, externally accessible components, code with a history of vulnerabilities, complex code, code that runs by default, and modules handling sensitive data. Many large OSS projects, including Chromium, Android, Qt, and Mozilla, as well as major industrial organizations, have adopted mandatory peer code review as part of their development processes [14,15].

Peer code review has received considerable attention for its role in improving software quality [17,29,35]. Edmundson et al. [23] demonstrated that although peer code review enhances software security, developers are not always able to identify all potential vulnerabilities. Although this practice improves security, some vulnerabilities remain undetected. This gap motivates a systematic examination of how peer code review contributes to security in OSS. Therefore, this research aims to conduct a Systematic Literature Review (SLR) examining peer code reviews to detect potential security vulnerabilities in open-source software.

3. Methodology

We followed the guidelines of Kitchenham and Charters [36] to conduct our SLR. The following subsections detail the process we followed.

¹ <https://zenodo.org/records/18945199>.

3.1. Defining the research questions

We want to dive deep into the latest research on how peer code reviews can help uncover security vulnerabilities in OSS. Conducting an in-depth analysis of peer code review research to identify security vulnerabilities within OSS is valuable for researchers seeking to understand existing exploration and identify unresolved questions.

Investigating the types of security vulnerabilities that can be identified during code review in OSS is essential to mitigate potential security risks at an early stage of development [37]. Delays in detecting and resolving security vulnerabilities increase their associated costs over time [38]. Hence, early identification and resolution of vulnerabilities through meticulous code reviews can significantly reduce the overall cost of developing secure software. Therefore, the first research question is:

RQ1 What types of potential security vulnerabilities can be identified or missed during peer code review in OSS?

OSS projects typically involve a diverse group of contributors, which can lead to inconsistent code quality and the introduction of potential security vulnerabilities. The role of security practices during code reviews in OSS is crucial for detecting and mitigating these vulnerabilities [9,22]. Although several guidelines and frameworks for secure coding practices have been suggested [9,25], there is a significant gap in empirical research that investigates the actual code review practices used in real-world scenarios. To provide a broad overview of security code review practices, the second research question is as follows.

RQ2 What are the current security practices during peer code review in OSS?

Security analysis tools, including static and dynamic analysis tools and vulnerability scanners, play a crucial role in the automated detection of security weaknesses, vulnerabilities, and code patterns. Previous studies have shown that incorporating these security analysis tools into the code review process can substantially improve the detection of potential vulnerabilities [39,40]. To explore the impact of security analysis tools on the code review process in OSS, the third research question is formulated as follows.

RQ3 How do security analysis tools contribute to improving OSS security during peer code reviews?

Investigating the challenges reviewers face in ensuring security during code review is crucial due to the unique dynamics of OSS development. Unlike proprietary software, OSS relies on voluntary contributions and peer review. The literature highlights that reviewers often struggle to identify complex vulnerabilities due to a lack of expertise, security knowledge gaps, cognitive overload, and time constraints [10,22,41]. Understanding these challenges is essential for developing effective methodologies, tools, and practices to enhance security-focused code reviews in OSS. So the fourth research question is as follows.

RQ4 What challenges and limitations do reviewers face in ensuring security during peer code reviews in OSS?

Identifying security vulnerabilities during code reviews is crucial for maintaining OSS security. The existing literature identifies several factors that enhance a reviewer's ability to identify vulnerabilities, including domain knowledge, security awareness, experience with secure coding practices, security knowledge, familiarity with the code base,

review time, and the integration of automated tools [10,15,22,42,43]. Despite existing insights, a deeper understanding of these enhancing factors is essential to improve the effectiveness of security-focused code reviews. Therefore, the fifth research question is as follows.

RQ5 What factors enhance a reviewer's ability to detect potential security vulnerabilities during peer code reviews in OSS?

3.2. Data source selection

We searched the following databases commonly used in SE SLRs [36,44].

- IEEE Xplore
- ACM Digital Library
- ScienceDirect
- Springer Link
- Scopus

3.3. Search string

To find relevant studies, we selected three terms to serve as the basis for the search. First, we chose "peer code review". Second, we chose "security" because it is the focus of our study. Third, we chose "open-source software" to exclude work focused on industrial software, which might employ different security practices. We then form the search strings by combining the keywords and their synonyms. We identified synonyms for each main term by reviewing prior related literature reviews and primary studies in software engineering and security. We refined and finalized the search terms and search string through discussion among the authors. This process ensured that our search terms captured the commonly used variations of the core concepts.

- **Term 1:-** "peer code review" OR "code review" OR "code inspection" OR "modern code review" OR "code checking" OR "code analysis"
- **Term 2:-** "security" OR "vulnerability" OR "threat" OR "attack" OR "vulnerable" OR "secure"
- **Term 3:-** "open-source software" OR "OSS" OR "FOSS" OR "free open-source software" OR "non-proprietary software" OR "FLOSS"

Because all three terms are necessary to identify the correct papers, we formed the search string as follows: (Term 1) AND (Term 2) AND (Term 3).

Search String: ("peer code review" OR "code review" OR "code inspection" OR "modern code review" OR "code checking" OR "code analysis") AND ("security" OR "vulnerability" OR "threat" OR "attack" OR "vulnerable" OR "secure") AND ("open-source software" OR "OSS" OR "FOSS" OR "free open-source software" OR "non-proprietary software" OR "FLOSS")

3.4. Selection criteria

We applied the inclusion and exclusion criteria listed in Table 1 during screening. For a study to be included, it must meet at least one inclusion criterion and not meet any exclusion criteria.

3.5. Primary study selection

We tailored our search string to match the specific syntax requirements of each digital library and executed the search in May 2023.

Table 1
Inclusion and exclusion criteria.

Inclusion criteria	Exclusion criteria
Articles that examine security vulnerabilities during peer code review in OSS	Duplicate articles
Articles that synthesize security practices applied during peer code review in OSS	Articles that are not peer-reviewed
Articles that discuss security analysis tools supporting vulnerability detection during code reviews in OSS	Articles not written in English
Articles that explore challenges reviewers face when identifying security vulnerabilities during peer code review in OSS	Abstracts, proposals, posters, or short papers
Articles that examine factors that enhance reviewers' ability to detect security vulnerabilities	Replication studies that do not provide new empirical evidence

Table 2
Search results by database.

Database	May 2023	May 2023–Sept 2024
IEEE Xplore	420	21
ACM	63	14
ScienceDirect	80	25
Springer	189	44
Scopus	189	22
Total retrieved	941	126
Duplicates	9	2
Unique results	932	124

Table 3
Study selection process.

Selection stage	May 2023	May 2023–Sept 2024
Unique search results	932	124
Title & abstract	416	24
Introduction & conclusion	40	7
Full-text	15	5

This search retrieved 941 records across the five databases (Table 2). Of those 941 papers, 9 appeared in more than one database. Once we removed the duplicates, 932 unique records remained for screening.

To maintain currency, we performed a follow-up search in September 2024 using the same search strategy. The updated search retrieved 126 additional records published between May 2023 and September 2024 (Table 2). Similar to the initial search, of the 126 new papers, 2 appeared in more than one database. After removing duplicates, 124 unique records remained for screening. We processed the initial and updated search rounds independently during duplicate removal and screening, and then combined the final screening results from both rounds.

Study selection proceeded in four phases: (1) removal of duplicates, (2) title and abstract screening, (3) introduction and conclusion screening, and (4) full-text review. Table 1 summarizes the inclusion and exclusion criteria. To ensure reliability, the second author independently applied the inclusion and exclusion criteria to a 20% random sample of papers in each phase. Inter-rater disagreements occurred in 15%, 8%, and 5% of papers during the second to fourth screening phases, respectively. All disagreements were resolved through discussion until the authors reached full agreement. Finally, we performed backward snowballing on the reference lists of the included studies. This process did not identify any additional papers meeting our criteria (see Table 3).

The 20 primary studies span 2009–2024, but most evidence is recent. Nineteen studies (95%) were published in 2013 or later, and among them 11 were published between 2020 and 2024. Table 7 in the Appendix reports publication year and author affiliations for each study. The affiliations cover multiple research groups across regions (e.g., Australia, Canada, Europe, and the United States), and we did not observe a single dominant group.

Most included studies are empirical (19/20, 95%). The remaining study is a systematic review that synthesizes empirical evidence related to modern code review practices. These studies mainly analyzed

OSS artifacts such as review comments, pull requests, patches, and review discussions mined from real-world open source projects and ecosystems (e.g., GitHub, Gerrit/Chromium OS, Qt, OpenStack, Mozilla Firefox, SmartSHARK, npm, and Python packages). Thus, our synthesis is grounded in evidence from real-world OSS development data, even though we do not directly mine repositories in this review.

3.6. Data extraction

The objective of this stage is to design data extraction forms. The first author then extracted all the data for this publication using that form. Then, the second author evaluated the extracted data to check for accuracy (see Table 4).

3.7. Data synthesis

For each primary study, we extracted findings relevant to each research question using the structured data extraction form described in Section 3.6. The extracted data included reported vulnerability types, security practices, tool usage, reviewer challenges, and enhancing factors, depending on which research questions the study addressed. This process produced a structured dataset of findings organized by research questions across all included primary studies.

We grouped extracted findings into recurring themes by identifying conceptually similar findings across studies using thematic aggregation. For RQ1, we mapped the reported vulnerability types and weaknesses to higher-level categories using the MITRE CWE-1000 Research Concepts view through a bottom-up aggregation process, as described in Section 4.1. This process enabled consistent aggregation and comparison of security issues reported across different studies. For RQ2 through RQ5, we iteratively compared and grouped conceptually similar findings across studies to derive higher-level themes and categories. Within each theme, we synthesized findings across studies by identifying recurring patterns, similarities, differences, and complementary observations reported in the literature. Both authors reviewed the synthesized themes and resolved disagreements through discussion to improve consistency and reduce interpretation bias.

4. Results

This section presents our findings based on the research questions. Throughout this section, we emphasize the key insights and discoveries. We also provide an in-depth analysis to contextualize these findings within the broader scope of our research objectives. By examining each question thoroughly, we offer a comprehensive understanding of the implications and significance of the results.

4.1. Classification of security issues

Most primary studies included in this review report security issues using CWE identifiers or vulnerability types aligned with the CWE taxonomy. In contrast, the Common Vulnerabilities and Exposures (CVE) database catalogs system-specific and publicly disclosed vulnerabilities

Table 4
Data extraction form.

Data item	Description
Data Extractor	Who extracts the data from the final set of papers
Data Checker	Who checks the extracted data to verify
Study Identifier	A unique digit identifier for a paper.
Title	The title of the paper.
RQ1.	What types of security vulnerabilities can be identified or missed during peer code review in OSS?
RQ2.	What are the current security practices during peer code review in OSS?
RQ3.	How do security analysis tools contribute to improving OSS security during peer code reviews?
RQ4.	What challenges and limitations do reviewers face in ensuring security during peer code reviews in OSS?
RQ5.	What factors enhance a reviewer's ability to detect security vulnerabilities during peer code reviews in OSS?

after deployment. Therefore, CVE is less suitable for characterizing early-stage detection activities such as code review.

As discussed in 2.1, this review adopts the MITRE CWE framework to classify security issues identified or missed during peer code review. The MITRE CWE taxonomy is organized hierarchically, grouping individual weakness types into progressively broader categories based on their underlying characteristics. The taxonomy defines parent–child relationships among weakness types, allowing researchers to analyze vulnerabilities at different abstraction levels. For example, CWE-79 (Cross-Site Scripting), CWE-77 (Command Injection), and CWE-89 (SQL Injection) are all subcategories of CWE-707 (Improper Neutralization).

To map the security issues reported in primary studies to high-level categories, we followed a bottom-up aggregation process using the CWE-1000 Research Concepts view [45]. For each CWE identifier reported in a primary study, we traversed the CWE hierarchy upward to identify its parent category, as described in the example above. In addition, some primary studies reported security issues using informal vulnerability labels (e.g., “injection flaws” or “XSS”) rather than explicit CWE IDs. We matched these issues to the closest CWE entry based on the study’s description. This aggregation does not alter the original classification reported in the primary studies while enabling higher-level analysis of recurring security issue patterns.

Throughout this paper, we use the term vulnerabilities in a practical sense to refer to potential vulnerabilities, security weaknesses, flaws, or defects identified during code review, consistent with the terminology used in the primary studies. However, our analysis and classification are grounded in the CWE framework to ensure clarity and consistency.

4.2. RQ1 What types of potential security vulnerabilities can be identified or missed during peer code review in OSS?

We separate the discussion of the identified potential vulnerabilities from the discussion of those that were missed.

4.2.1. High-level vulnerability types

To simplify the analysis, we consolidated the identified and missed vulnerability categories into 10 high-level categories. We performed this reduction based on the hierarchical classification in the official CWE taxonomy, which defines relationships among different types of weaknesses. For example, CWE-79 (cross-site scripting), CWE-77 (command injection), and CWE-89 (SQL injection) are all subcategories under CWE-707 (Improper neutralization). The following text explains each high-level category.

Improper Control of a Resource Through its Lifetime (CWE-664). This vulnerability occurs when a system fails to properly manage a resource from its creation to its release. This category of vulnerabilities encompasses issues such as buffer overrun, information leakage, format string problem, improper synchronization, exposure of resource to wrong sphere, improper initialization, operation on a resource after expiration or release, and uncontrolled resource consumption, etc.

Improper Neutralization (CWE-707). This vulnerability occurs when a system fails to correctly handle, validate, or sanitize user input, allowing attackers to inject malicious data. This vulnerability can lead to injection-based vulnerabilities, such as cross-site scripting (XSS), cross-site request forgery (CSRF), SQL injection, and command injection.

Improper Access Control (CWE-284). This vulnerability occurs when a system either fails to restrict or incorrectly restricts access to a resource from an unauthorized actor. This vulnerability type includes issues related to authentication, authorization, and accountability.

Improper Adherence to Coding Standards (CWE-710). This vulnerability occurs when a product fails to adhere to established coding rules or development standards. This vulnerability type includes the use of potentially dangerous functions, third-party vulnerable packages, security misconfiguration, and documentation.

Improper Check or Handling of Exceptional Conditions (CWE—703). This vulnerability occurs when a product fails to anticipate or handle exceptional conditions that occur during normal operation. This vulnerability type includes crashes, catching all exceptions, and improper exception handling.

Insufficient Control Flow Management (CWE-691). This vulnerability occurs when code fails to properly control its execution path, allowing unexpected changes during program execution. This vulnerability type includes race conditions and uncontrolled recursion.

Protection Mechanism Failure (CWE-693). This vulnerability occurs when a system fails to use or improperly utilizes a protection mechanism to defend against attacks. This vulnerability type includes failing to protect network traffic and encryption.

Incorrect Calculation (CWE-682). This vulnerability occurs when a system performs a calculation that generates incorrect or unintended results. This error may lead to issues such as incorrect resource allocations, improper privilege assignments, or inadequate resource management.

Weakness that Affect System Process (CWE-634). This vulnerability occurs when defects in hardware and software negatively impact system-level operations. Note that, now this CWE category is listed as deprecated on cwe.mitre.org.

C++ Catastrophes. This vulnerability does not have an assigned CWE identifier, as it refers to critical problems specific to the C++ language that can lead to severe security flaws.

4.2.2. Identified vulnerabilities

The analysis of RQ1 shows that peer reviews in OSS are capable of uncovering a wide range of vulnerabilities, which we organized into high-level categories based on the CWE classification. Across the 20 primary studies, we identified 46 distinct vulnerability types detected during code review, with a total reported frequency of 1789 instances spanning ten high-level CWE categories.

The most prevalent category was *Improper Control of a Resource Through its Lifetime* (CWE-664), accounting for 587 instances. Common vulnerabilities under this category included buffer overrun, improper synchronization, improper resource shutdown or release, and resource leak. The second most prevalent category was *Insufficient Control Flow Management* (CWE-691), with 464 instances, primarily involving race conditions and uncontrolled recursion. The third most prevalent category was *Improper Check or Handling of Exceptional Conditions* (CWE-703), comprising 203 instances, most often related to crashes, improper checks, and catching all exceptions. Moderately frequent categories included *Improper Access Control* (CWE-284), which was reported in 152 instances, covering vulnerabilities such as improper authentication and too much privileges. *Incorrect Calculation* (CWE-682) was found in 151 cases, typically integer overflows and incorrect calculations. *Improper Neutralization* (CWE-707) was also notable, with 126 cases, including client-side cross-site scripting (XSS), SQL injection, code injection, and improper input validation. *Improper Adherence to Coding Standards* (CWE-710) was noted in 71 instances, encompassing the use of potentially dangerous functions, inadequate documentation, and the use of vulnerable packages. The less frequently identified categories were *Protection Mechanism Failures* (CWE-693), with 26 instances, mainly encryption weaknesses. A specialized category, the language-specific C++ *Catastrophe*, was observed in 8 instances, and *Weaknesses Affecting System processes* (CWE-634) appeared in just one instance.

A complete list of identified vulnerabilities and their frequencies is provided in Table 5 in the Appendix section.

4.2.3. Missed vulnerabilities

Our findings reveal that peer code review in OSS frequently overlooks a diverse set of security vulnerabilities. Across the 20 primary studies, we identified 38 distinct vulnerability types that were missed during peer code review, with a total reported frequency of 461 instances spanning the same ten high-level CWE categories.

The most frequently missed category was *Improper Control of a Resource through its Lifetime* (CWE-664) with 180 instances. The vulnerabilities under this category were operation on a resource after expiration or release, and exposure of the resource to the wrong sphere. The second most common category was *Improper Neutralization* (CWE-707), comprising 102 instances. The most commonly overlooked vulnerabilities in this category were improper input validation, client-side cross-site scripting (XSS), and data-processing errors. The third most frequently overlooked category is *Improper Access Control* (CWE-284), with 65 instances, primarily involving general improper access control and too much privileges. The special category of C++ *Catastrophe* vulnerabilities accounted for 33 missed cases. *Improper check or handling of exceptional conditions* (CWE-703) was missed in 22 instances, while *incorrect calculation* (CWE-682) occurred in 20 instances. Both *Insufficient Control Flow Management* (CWE-691) and *Protection Mechanism Failure* (CWE-693) accounted for 17 missed instances. The former was predominantly associated with race conditions, while the latter was related to insufficient verification of data authenticity. *Improper Adherence to Coding Standards* (CWE-710) appeared in 4 instances, linked to the use of potentially dangerous functions. Finally, *Weaknesses Affecting System processes* (CWE-634) was the least frequent, with only one instance recorded.

A complete list of identified vulnerabilities and their frequencies is provided in Table 6 in the Appendix section.

4.3. RQ2 What are the current security practices during peer code review in OSS?

We grouped the identified practices into the categories described below.

4.3.1. Reviewer involvement

The involvement of multiple reviewers in peer code reviews is a common practice that can significantly enhance the detection of security vulnerabilities in OSS, particularly when reviewers have relevant security knowledge [17,42]. Code reviews in the Chromium open-source project found that reviews with more than two reviewers were more effective in identifying security vulnerabilities [42]. Similarly, prior studies [35,46] have observed that a smaller number of reviewers is associated with a higher risk of overlooking security vulnerabilities. However, the relationship between the number of reviewers and security outcomes is not consistently supported across the literature. Meneely et al. [10] observed that vulnerable files often had more reviewers, many of whom lacked security knowledge. Paul et al. [15] found reviewers who perform an excessive number of reviews may become less cautious and miss security vulnerabilities. Thompson et al. [12] found no statistically significant relationship between the number of reviewers and the identification of security vulnerabilities. These findings suggest that an optimal number of reviewers, balanced with security knowledge, is critical for achieving effective security outcomes.

4.3.2. Reviewer expertise and code familiarity

Reviewer expertise, familiarity with the codebase, and project context have a significant impact on vulnerability detection in open-source software. Reviews conducted by developers with prior experience in identifying or fixing vulnerabilities are more likely to detect critical vulnerabilities [10,47]. In addition, increased reviewer expertise and closer collaboration between reviewers and authors reduces the likelihood of post-release security vulnerabilities [10]. In contrast, reviews involving participants with limited security knowledge and unfamiliarity with the relevant code are more likely to miss vulnerabilities [17, 46].

However, the relationship between experience and vulnerability detection is not entirely straightforward. One study reported that highly experienced developers were often responsible for introducing vulnerable code [22]. However, code changes by less experienced authors are significantly more likely to be vulnerable, ranging from 1.5 to 24 times higher than those by experienced authors [22]. Another study found no clear relationship between the reviewer's level of experience and detection accuracy, suggesting that specific security knowledge may outweigh general experience [23]. These findings suggest that while reviewer expertise and project familiarity are essential for detecting security vulnerabilities during code review, experience alone is insufficient.

4.3.3. Code change characteristics

The size and complexity of code changes influence the likelihood of security issues being detected during review. Smaller changes and clearly articulated review requests tend to draw greater reviewer participation and scrutiny [47]. In contrast, larger code modifications and high code churn increase the risk of introducing vulnerabilities, as their complexity can overwhelm reviewers [10,22]. Additionally, modified files are more prone to vulnerabilities than new ones, necessitating heightened scrutiny for alterations to existing code [22]. These insights suggest that OSS projects can enhance security by promoting concise and well-documented code changes.

4.3.4. Addressing vulnerabilities

While peer code review is a widely adopted practice for improving software quality and identifying potential vulnerabilities, empirical evidence suggests that vulnerability detection often receives less emphasis than expected during the review process. Bacchelli and Bird [29] found that code reviews tend to focus more on general quality concerns than on security-specific issues. Charoenwet et al. [48] further support this pattern by showing that developers identified and discussed coding weaknesses significantly more frequently than security vulnerabilities,

by a factor ranging from 21 to 33.5. These findings indicate that although security is a recognized objective, it is frequently overshadowed by other review priorities.

When code review identifies security issues, the developers' responses vary considerably. One study [40] reported that 54.96% of security-related concerns raised during code review were addressed through fixes or mitigations. Nonetheless, a substantial proportion of issues (28.65%) were assessed as low risk and not prioritized for immediate resolution. Additionally, a notable percentage of issues remained unresolved (8.18%) or received no response at all (4.67%). Similarly, another study [48] found that while developers often attempted to resolve identified concerns within the same pull request, 30% to 36% of these issues were acknowledged and merged without resolution, and 15% to 26% were either dismissed or left unaddressed.

The degree of reviewer engagement in guiding the resolution of security issues also varies. According to one study [49], over half of the reviewers (53.8%) anticipated that developers would take responsibility for addressing the reported vulnerabilities. In most of these instances (86.6%), the reviewers provided explicit guidance, including suggested fixes or code snippets. However, in 13.4% of the cases, reviewers identified issues without offering actionable advice. Furthermore, in 39% of the cases, reviewers noted the presence of a vulnerability without indicating any next steps, and in 7.2% of cases, reviewers explicitly recommended ignoring the issue, often citing its perceived insignificance. Collectively, these findings highlight inconsistencies in how security concerns are addressed during code review. These patterns suggest a need for more structured practices and clearer expectations to support the consistent handling of security vulnerabilities in the peer review process.

4.3.5. Human and organizational factors

Human and organizational dynamics have a significant impact on security practices in OSS code reviews. A study by Braz et al. [41] provided the following insights. Developers recognize the importance of security but often give it a lower priority unless explicitly prompted otherwise, and sometimes assume their applications are inherently secure. Additionally, the majority of organizations fail to provide adequate security training, yet they expect developers to address security issues during code reviews. Moreover, developers' efforts in security contributions are often unrecognized, which may further reduce the focus on security during code reviews. Another study [43] demonstrated that explicitly instructing reviewers to focus on security during code review increased the likelihood of vulnerability detection by a factor of eight. This insight suggests that without support from the organization and cultural emphasis on security, even skilled reviewers and good practices may not be enough. It highlights the need for better training and recognition in open-source projects.

4.4. RQ3 How do security analysis tools contribute to improving OSS security during peer code reviews?

Security analysis tools, particularly static application security testing (SAST) tools, significantly enhance the security of open-source software (OSS) during code reviews. These tools assist reviewers by flagging potentially vulnerable code segments, thereby reducing the manual effort required from reviewers and allowing them to focus on more complex aspects of the code. One study [39] demonstrated that prioritizing functions based on SAST warnings improves the identification of vulnerabilities, leading to a 12% increase in precision, a 5.6% increase in recall, and a 13% reduction in false positives compared to non-prioritized reviews. This result suggests that integrating the output of security analysis tools into review workflows significantly reduces the burden on human reviewers and improves the overall quality of the review process.

The integration of security tools into development workflows further supports early detection of common vulnerabilities. For instance,

individual SAST tools were capable of detecting approximately 52% of known vulnerabilities in modified functions, and combining multiple tools increased detection rates by an additional 26% [39]. In addition, several projects have adopted automated tools in continuous integration (CI) pipelines to support security checks during the review process [40]. One such example is the Marked project in the npm ecosystem, which uses the vuln-regex-detector tool to identify ReDOS vulnerabilities [50]. These cases demonstrate how the integration of security tools can reliably identify security issues early within the development workflow.

Despite these benefits, security analysis tools have notable limitations. The evaluated SAST tools generated a large number of irrelevant warnings, with 76% being false positives. Furthermore, they failed to produce any warning for 22% of vulnerable code changes (VCCs) [39]. These results highlight the significant limitations of current SAST tools. Therefore, the effective use of static analysis tools requires selecting tools that are aligned with the project's specific security requirements and development context, as well as supplementing the tool output with human judgment and manual review.

4.5. RQ4 What challenges and limitations do reviewers face in ensuring security during peer code reviews in OSS?

Peer code reviews play a vital role in improving security in OSS. However, reviewers face several challenges that limit their effectiveness. This section outlines the key challenges identified in prior studies.

4.5.1. Insufficient security knowledge and expertise

A significant challenge to ensuring security during code reviews in OSS is the reviewers' insufficient security knowledge and lack of secure coding practices. Prior research consistently highlights that developers often lack security knowledge, secure coding practices, and have limited familiarity with common vulnerability patterns [10,22,41]. One study [22] found that different developers introduce similar types of security vulnerabilities, such as unsafe method usage, failure to sanitize inputs, TOCTOU race conditions, and integer arithmetic overflows, across code changes. These vulnerabilities can be mitigated by acquiring security knowledge and adopting secure coding practices. Insufficient security training is also a significant challenge in effectively ensuring security during code reviews [41]. Furthermore, reviewers frequently encounter challenges in ensuring security, particularly when they have limited security experience and are unfamiliar with collaborators and the projects [10,17,46].

4.5.2. Time constraints and review fatigue

Time constraints and high review volumes significantly impact reviewers' ability to focus on security. Time constraints are particularly prevalent, leading to superficial review efforts and reduced attention to security considerations [15,41]. In addition, high review volumes lead to review fatigue, thereby reducing the likelihood of detecting security vulnerabilities [15]. Reviewing a high volume of code under time constraints can lead to review fatigue, which reduces a reviewer's ability to effectively identify security vulnerabilities.

4.5.3. Contextual and practical constraints

Reviewers face several contextual and practical constraints that limit their ability to identify security vulnerabilities. Code changes that affect multiple directories and files significantly expand the review scope, making it more difficult for reviewers to detect security vulnerabilities [15]. Many reviewers lack a comprehensive understanding of the application under review, including the behavior of its components, third-party library dependencies [29,41]. These challenges make it difficult to assess whether a given code change might expose the system to new threats. In addition, the broad and evolving landscape of potential vulnerabilities, combined with diverse and subtle attack patterns, further complicates the identification of security vulnerabilities during code reviews [41].

4.5.4. Human factors and process limitations

Human factors, combined with inherent limitations in the code review process, make it more challenging for reviewers to identify vulnerabilities. For example, some vulnerabilities only become apparent during code execution, making them difficult to spot through manual peer code review alone [41]. Human factors, such as limited security awareness, reviewers' mindset, and cognitive load, also hinder the effectiveness of security-focused code reviews. Reviewers may not prioritize or recognize the importance of security concerns unless explicitly directed to do so [29,41].

4.6. RQ5 What factors enhance a reviewer's ability to detect potential security vulnerabilities during peer code reviews in OSS?

Understanding the factors that enhance a reviewer's ability to detect security vulnerabilities is essential for improving the effectiveness of peer code reviews in OSS. This section presents the key factors identified in the literature.

4.6.1. Security knowledge and expertise

The reviewer's security knowledge strongly influences their ability to detect security vulnerabilities during code review. One study [41] found that reviewers with security knowledge acquired through formal education, security training, or participation in workshops demonstrate greater proficiency in identifying vulnerabilities during code reviews. Security expertise is a critical factor in effective vulnerability detection during code review. Reviewers with a strong foundation in security are better equipped to identify potential vulnerabilities [10,47]. This expertise encompasses both broad knowledge of security principles and specific experience in vulnerability identification [10].

4.6.2. Review background and codebase familiarity

Familiarity with the codebase and prior review experience significantly enhances a reviewer's ability to detect security vulnerabilities. Reviewers with previous experience reviewing a specific file or with a history of collaboration with the code author are more likely to identify security issues [10,15]. Familiarity with the codebase and active engagement in the review process help reviewers better understand the context and impact of code changes, which significantly improves their ability to detect security vulnerabilities [10,47].

4.6.3. Mental attitude and security awareness

The reviewer's mental attitude and focus toward security practices also shape their effectiveness in identifying vulnerabilities. One study [43] found that developers who are explicitly instructed to prioritize security concerns during code review are more successful in identifying vulnerabilities compared to those who are not. In addition, their results found that the mental attitude of reviewers and consistent adherence to secure coding guidelines play a significant role in detecting security vulnerabilities during code reviews. Similarly, another study [41] further emphasizes that organizational culture and guidelines that promote security awareness can reinforce this mindset, enhancing reviewers' vigilance and reducing the likelihood of overlooking common vulnerabilities.

4.6.4. Training, support, and review environment

Organizational and contextual factors substantially influence the effectiveness of security-focused code reviews. One study [41] emphasizes that participation in security-specific training or workshops enhances reviewers' ability to identify vulnerabilities by providing relevant knowledge and up-to-date techniques. The same study showed that access to security expert support, resources, and rewards for vulnerability detection can motivate reviewers to evaluate code with a security-oriented perspective. These findings underscore the importance of a structured and supportive review environment, characterized by formal training opportunities, expert guidance, reward, and adequate time allocation to enhance the quality and reliability of security assessments during code reviews.

4.6.5. Review duration and engagement

The amount of time spent on a code review and the reviewer's level of engagement play an essential role in identifying security vulnerabilities. Longer review sessions provide reviewers with more opportunities to examine the code in detail, thereby increasing the likelihood of identifying security vulnerabilities [15]. However, extended reviews can lead to review fatigue, which may reduce effectiveness. In addition to time, the quality of reviewer engagement is also essential. Reviewers who are actively involved and thorough in their assessments are more likely to detect security vulnerabilities [47]. These findings suggest that sufficient time and focused effort are essential to conducting effective security-oriented code reviews.

5. Discussion

This section synthesizes the findings from our systematic literature review to provide a comprehensive interpretation of the results. This discussion also outlines practical implications and future directions to enhance the effectiveness of peer code reviews as a security assurance mechanism. In addition, recent advances in AI-assisted code review introduce new opportunities and challenges. We discuss these emerging trends in Section 5.6.

5.1. RQ1 What types of potential security vulnerabilities can be identified or missed during peer code review in OSS?

The findings from RQ1 reveal clear patterns in reviewers' strengths and limitations when identifying security vulnerabilities during OSS peer code reviews. Reviewers are generally effective at detecting vulnerabilities that are simple, well-known, visible, and localized in the code change. Examples include buffer overruns, race conditions, crashes, XSS, and the use of well-known dangerous functions. These vulnerabilities are widely documented in secure coding guidelines that align closely with developers' programming knowledge and are easier to spot through code review. Conversely, vulnerabilities that are complex, subtle, semantic, context-dependent, and necessitate code execution or a thorough understanding of code context, such as improper input validation, improper access control, and resource lifecycle errors, were less likely to be detected. These issues often require deeper reasoning about broader system behavior, data propagation, and privilege boundaries, which are difficult to infer from isolated code snippets. These findings align with the results of the prior study by Paul et al. [15], which highlight the limitations of manual reviews for semantic or intricate flaws.

To enhance effectiveness, OSS communities could prioritize security knowledge and training, structured review checklists, and integration of automated tools to complement human oversight. Such measures can help reduce overlooked vulnerabilities and strengthen the overall security of OSS projects.

5.2. RQ2 What are the current security practices during peer code review in OSS?

The results for RQ2 indicate that effective security practices in OSS peer code reviews depend on the alignment of technical practices and human-organizational supports, including reviewer involvement, expertise, security knowledge, code familiarity, code change characteristics, and managerial support. Reviews are most effective when they involve reviewers who not only possess security expertise but are also familiar with the codebase and project context. These reviewers are better able to interpret code changes accurately and identify security issues, especially when the changes are small, well-documented, and clearly scoped. Such conditions help focus the reviewer's effort and reduce the risk of overlooking vulnerabilities.

However, the results also reveal essential gaps and inconsistencies in current practices. Code quality or coding style concerns often

overshadow security issues during code reviews. Even when reviewers identify vulnerabilities, their responses are inconsistent. Reviewers often classify many vulnerabilities as low risk, leave them unresolved, or do not follow up correctly during the review process. This approach to handling vulnerabilities during reviews reflects a lack of structured processes for prioritizing and resolving security issues. Moreover, human and organizational factors have a significant impact on security practices in OSS code reviews. Developers often deprioritize security unless explicitly asked to focus on it. Many organizations expect reviewers to address security, but do not provide training or recognition for doing so.

Overall, these patterns highlight the need for OSS communities to foster a security-centric culture. This need includes balancing the number and expertise of reviewers, ensure project and code familiarity, managing code change size, structured protocols for prioritizing and resolving security issues, providing security training, and promoting a security oriented mindset.

5.3. RQ3 How do security analysis tools contribute to improving OSS security during peer code reviews?

The integration of security analysis tools, especially SAST, plays a key role in supporting the identification of security issues in OSS during peer code reviews. These tools help reduce the cognitive load on human reviewers by automatically flagging potentially vulnerable code patterns. This automated detection allows reviewers to focus their attention on more complex or contextual issues. However, the results also reveal that tool-based approaches alone are insufficient. Tools often miss specific vulnerabilities, produce high false positives, and lead to reviewer overload due to excessive false alerts. These limitations underscore the importance of carefully selecting tools that align with a project's security requirements. Overall, these findings highlight that security analysis tools are most effective when used in conjunction with a human reviewer. Embedding tools in review workflows can improve efficiency and coverage, but their success ultimately relies on knowledgeable reviewers who can contextualize tool findings and make informed security decisions.

5.4. RQ4 What challenges and limitations do reviewers face in ensuring security during peer code reviews in OSS?

The findings for RQ4 reveal that reviewers face several interconnected challenges that hinder their ability to ensure security during OSS code reviews. The most significant challenge is the reviewers' limited knowledge and expertise in security. Empirical studies demonstrate that many reviewers lack sufficient familiarity with secure coding practices and common vulnerability patterns, which increases their likelihood of overlooking critical security issues. Limited security training opportunities and unfamiliarity with project-specific contexts and collaborators worsen the problems. In addition, review fatigue and time constraints often force reviewers to prioritize functionality and coding style over security issues.

Human and organizational factors, as well as contextual and process limitations, further complicate the reviewers' ability to identify security vulnerabilities. Reviews become more complex when code changes span multiple directories, involve third-party libraries, or modify large subsystems. Human and cultural factors, such as reviewer mindset, limited awareness, and high cognitive load, also reduce their likelihood of detecting subtle vulnerabilities. Moreover, reviewers may not prioritize security concerns unless they are explicitly instructed to do so, reflecting a broader cultural tendency to treat security as secondary to other review goals.

In summary, reviewers face challenges that include a lack of security knowledge, human and cognitive limitations, organizational shortcomings, and practical constraints within the review process. Addressing these challenges requires security focused training to enhance

reviewer knowledge, structured review practices to manage workload effectively, the integration of automated tools to complement human judgment, and a stronger organizational emphasis on security during code reviews.

5.5. RQ5 What factors enhance a reviewer's ability to detect potential security vulnerabilities during peer code reviews in OSS?

The findings for RQ5 show that a reviewer's ability to detect security vulnerabilities in OSS depends on both technical skills and the environment in which reviews take place. Reviewers who possess strong security knowledge and experience and have attended security training and workshops are more effective at spotting vulnerabilities. Familiarity with the project and collaborators is also essential. Reviewers who understand the context and history of the code are better able to identify security issues. In addition, a security-focused attitude and awareness significantly affect security outcomes. Reviewers who approach code review with an explicit focus on security or are prompted by organizational guidelines to prioritize security concerns are more likely to identify vulnerabilities.

Organizational and contextual factors further enhance reviewer effectiveness. Access to security training, support from security experts, and recognition or rewards for finding vulnerabilities all motivate reviewers to take security issues more seriously. In addition, spending sufficient time and maintaining focus during code reviews enables reviewers to examine changes in depth, thereby enhancing their ability to detect security vulnerabilities. However, excessive review duration can lead to review fatigue and negatively impact performance.

In summary, a reviewer's ability to detect security vulnerabilities in OSS depends on security knowledge and experience, code-base familiarity, security-focused attitudes and awareness, organizational support, and sustainable review practices. Future research could investigate these factors in various OSS settings and examine how combining reviewer expertise with AI-based support impacts their effectiveness.

5.6. Emerging trends: AI-augmented code review for security in OSS

This review focuses on manual peer code review rather than automated or AI-powered code review. However, recent advances in generative AI, particularly large language models (LLMs), have opened new opportunities to support software engineering tasks. LLMs have demonstrated strong capabilities in code understanding, including code generation, summarization, vulnerability pattern recognition, program repair, and assistance with static analysis [51–54]. These capabilities have motivated growing interest in AI-assisted techniques that augment traditional peer code-review workflows.

From a security perspective, AI-based approaches can address several limitations identified in this SLR. For example, our findings in RQ1 show that reviewers often miss vulnerabilities that require contextual reasoning and broader execution context. Similarly, RQ4 highlights the impact of limited expertise, time pressure, and cognitive load on vulnerability detection. AI-assisted analysis can help mitigate these challenges by automatically identifying suspicious patterns, prioritizing high-risk code regions, and reducing reviewer workload. In this way, AI support aligns with the needs identified in this study, particularly in assisting reviewers with limited security expertise.

Despite these opportunities, current evidence suggests that AI techniques are not yet reliable as standalone solutions. Recent empirical studies indicate that general-purpose LLMs struggle with deep semantic reasoning, data-flow analysis, and project-level context [55,56]. Moreover, LLMs may generate hallucinated or verbose outputs, which raises concerns about trust, explainability, and reliability in security-critical environments [57]. Privacy, bias, and ethical considerations also limit

the direct use of these methods in OSS settings, where code may be sensitive or confidential.

Overall, these findings reinforce that AI-assisted tools can be viewed as a complementary aid in security-focused code review, but they cannot replace human judgment. AI-assisted tools can support vulnerability detection by guiding attention and improving prioritization, while human reviewers remain essential for reasoning and security decision-making. Future research should empirically evaluate the impact of AI-assisted review in real OSS repositories and explore hybrid human–AI workflows that improve both effectiveness and efficiency.

5.7. Implications of OSS development context

The findings of this review highlight several factors that affect the effectiveness of security issues detection during code review. However, the role of the OSS development context itself remains underexplored in the existing literature. OSS projects are often characterized by geographically distributed contributors, asynchronous collaboration [58–60], varying levels of expertise [61], and diverse governance and sponsorship models [62,63]. These characteristics may influence reviewers' ability to understand system architecture, project history, and design rationale. As discussed in Section 4.6.2, familiarity with the codebase is a key factor in detecting security issues. However, the included studies rarely examine how these factors are shaped by OSS-specific development environments.

The distributed and asynchronous nature of OSS development also has implications for tool support. As discussed in Section 4.4, security analysis tools can provide a baseline guidance across diverse contributors. However, many studies evaluated these tools using OSS repositories mainly as empirical testbeds rather than explicitly analyzing how OSS-specific characteristics influence tool effectiveness.

Similarly, organizational and governance aspects discussed in Sections 4.6.4 and 5.2 may differ across OSS settings. Community-driven projects often rely on informal guidelines and experienced maintainers, whereas sponsored OSS projects may provide structured policies, training, rewards and incentives, and security expertise. The included studies rarely distinguish between these governance models. Therefore, future research should investigate how governance structure, contributor distribution, and collaboration practices influence security-focused code review in real-world OSS environments.

6. Threats to validity

This section describes the threats to validity of the study and how we addressed them, where possible.

6.1. Search strategy and study selection

The quality and coverage of an SLR depend heavily on the search strategy and study selection process. To identify relevant studies, we designed the search string using terms derived from prior work on software security, peer code review, and OSS development. We refined the search string through pilot searches and discussions between the authors to improve coverage and reduce bias.

We searched multiple digital libraries covering software engineering journals, conferences, and workshops. Given that there is no single standard venue for publishing this type of work, some relevant studies may still have been missed. However, we additionally performed backward snowballing to mitigate this threat as much as possible.

We also used predefined inclusion and exclusion criteria during study selection. Two authors independently screened a subset of studies at different stages of the review process and resolved disagreements through discussion. Despite these efforts, some relevant studies may still have been missed due to differences in terminology, database indexing limitations, or publication availability.

6.2. Security vulnerability classification

Most primary studies reported potential vulnerabilities or security issues using CWE identifiers. To maintain consistency across studies, we mapped the reported vulnerabilities to high-level CWE categories using the official MITRE taxonomy [28]. However, some studies described vulnerabilities only by name and did not explicitly provide CWE identifiers. In those cases, the mapping process required interpretation, which may introduce classification bias. To reduce this risk, both authors reviewed the mappings and resolved disagreements through discussion. Nevertheless, some degree of subjectivity may remain in the classification of vulnerabilities across studies.

6.3. Scope and generalizability

This review focuses on peer code review in OSS projects. Many included studies examined large and well-established OSS projects such as Chromium, Qt, OpenStack, Mozilla, and npm packages. Therefore, the findings may not generalize to smaller community-driven OSS projects or proprietary software environments. In addition, OSS projects differ in governance structure, contributor expertise, sponsorship, and development practices. Most primary studies did not explicitly distinguish among these contextual factors, which may limit the applicability of the findings across different OSS ecosystems.

6.4. Reliability

Reliability concerns whether the review process is sufficiently transparent and consistent for another research team to follow the same procedure. To improve consistency and reliability, we used structured data extraction forms and systematically synthesized findings across studies. Two authors participated in the study selection and vulnerability classification process and resolved disagreements through discussion. The documented review protocol, independent screening process, and transparent reporting of decisions improve the replicability of the review and increase confidence in the stability of the findings.

7. Conclusion

This paper presents the first SLR on the role of peer code review in identifying security vulnerabilities in OSS. While peer code review has long been recognized as a valuable practice for improving code quality, studies of its effects on security are relatively recent. By synthesizing findings from 20 primary studies, this review provides a comprehensive overview of the vulnerabilities identified and missed during code reviews, the practices and tools that support security detection, and the challenges and factors influencing reviewer effectiveness.

The results show that peer code reviews are effective in detecting common, localized vulnerabilities, particularly those that require understanding only a few lines of code. However, reviewers frequently overlook more complex vulnerabilities that require reasoning about execution context, resource lifecycles, or system-level interactions. Security practices emphasize balancing the number of reviewers, their expertise, project familiarity, and prioritizing high-risk changes. However, inconsistencies in addressing identified security concerns highlight gaps in security practices in the code review process.

Security analysis tools, particularly SAST integrated into continuous integration (CI) pipelines enhance vulnerability detection. Their effectiveness is maximized when used in combination with human judgment that suggests the importance of hybrid human–tool approaches.

Reviewers face challenges such as insufficient security knowledge, training, limited expertise, time pressure, and difficulties understanding complex systems and third-party dependencies. Human and organizational factors like security awareness, lack of explicit guidance, and cultural tendencies further contribute to the low prioritization of security during code reviews. The factors that influence reviewer

effectiveness depend on both technical and organizational conditions, including security knowledge, attitudes, familiarity with the code base, training, structured review practices, and organizational support.

The peer code review process alone cannot guarantee complete security. However, when combined with structured checklists, automated analysis tools, and a security-focused review culture, proper acknowledgment of effort can substantially reduce post-release vulnerabilities in OSS. For researchers, this SLR establishes a baseline for evaluating the security impact of peer code reviews, bridging gaps between code quality and vulnerability mitigation. Future research should focus on developing and integrating AI-assisted security analysis tools into code review workflows, reviewer-centered guidelines, and exploring approaches to reduce the challenges that reviewers face during code reviews.

CRedit authorship contribution statement

Md Niaz Morshed: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Data

curation, Conceptualization. **Jeffrey C. Carver:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

See [Tables 5–7](#).

Data availability

Data will be made available on request.

Table 5
Identified potential security vulnerabilities.

CWE category (High level)	Vulnerability name	Citations	Freq.
Improper Control of a Resource Through its Lifetime (CWE-664)	Buffer overruns	[22,40,42,49,64,65]	118
	Information leakage	[22,42,48,66]	16
	Format string problems	[22,42,49,64]	15
	Use of magic URLs/Predictable cookies	[42]	1
	Incorrect type conversion or cast	[15,42]	7
	Improper control of a resource in its lifetime	[42]	1
	Regular Expression DoS (ReDoS)	[40,66]	23
	Sensitive data exposure	[40]	10
	Deadlock	[22,40,49]	31
	Denial of Service (DoS)	[22,40,49,66]	33
	Memory leak	[22,65]	2
	Resource leak	[49]	59
	Use after free	[49]	8
	Improper synchronization	[15]	109
	Exposure of resource to wrong sphere	[15]	15
	Improper initialization	[15]	5
	Improper resource shutdown or release	[15]	71
	Operation on a resource after expiration or release	[15]	3
Insufficient Control Flow Management (CWE-691)	Improper restriction of operations within bounds of memory buffer	[15,48]	53
	Uncontrolled resource consumption	[48]	7
Improper Neutralization (CWE-707)	Race conditions	[15,22,40,42,48,49]	456
	Uncontrolled recursion	[48]	8
	Client-side XSS	[22,23,40,42,49,66]	72
	SQL injection	[22,23,40,49]	27
	Remote code injection	[40,66]	9
	Improper input validation	[15,40,66]	13
Improper Access Control (CWE-284)	Cross-Site Request Forgery (CSRF)	[22,23,49]	3
	Command injection	[22,49]	2
	Too much privilege	[42]	5
	Weak password-based systems	[42]	1
Improper Adherence to Coding Standards (CWE-710)	Privilege/Sandbox issues	[42]	2
	Improper authentication	[40,48,66]	43
	Improper access control	[15,22,40,49]	101
	Documentation issues	[40]	14
Improper Check or Handling of Exceptional Conditions (CWE-703)	Vulnerable packages	[40]	3
	Security misconfiguration	[22]	1
	Potentially dangerous functions	[15]	53
	Catching all exceptions	[42]	3
Protection Mechanism Failure (CWE-693)	Crash	[22,49]	142
	Improper exception handling	[15]	58
Incorrect Calculation (CWE-682)	Failing to protect network traffic	[42]	2
	Encryption	[48,49]	24
C++ Catastrophes (CWE-634 (Deprecated))	Integer overflows	[15,22,42,49]	86
	Incorrect calculation	[15]	65
C++ Catastrophes (CWE-634 (Deprecated))	C++-specific vulnerabilities	[42]	8
	Weaknesses affecting system processes (IPC)	[42]	1

Table 6

Missed or overlooked potential security vulnerabilities.

CWE category (High level)	Vulnerability name	Citations	Freq.
Improper Control of a Resource Through its Lifetime (CWE-664)	Buffer overruns	[42,65]	18
	Information leakage	[42]	1
	Format string problems	[42]	3
	Magic URLs/Predictable cookies	[42]	1
	Incorrect type conversion or cast	[15,42]	18
	Improper control of a resource in its lifetime	[42]	2
	Improper synchronization	[15]	13
	Improper resource shutdown or release	[15]	4
	Operation after expiration or release	[15]	62
	Improper restriction within memory bounds	[15]	16
	Exposure of resource to wrong sphere	[15]	35
	Session expiration	[67]	1
	Memory leak	[65,67]	3
	Dangling pointer	[65]	1
	Uninitialized memory read	[65]	1
	Incorrect memory allocation	[65]	1
Protection Mechanism Failure (CWE-693)	Failing to protect network traffic	[42]	4
	Insufficient verification of data authenticity	[15]	13
Improper Access Control (CWE-284)	Too much privilege	[42]	5
	Weak password-based systems	[42]	1
	Improper access control	[15]	58
	Authentication	[67]	1
Improper Neutralization (CWE-707)	XSS (client side)	[42]	12
	Command injection	[42]	7
	XSS (server side)	[42]	1
	Improper input validation	[15]	72
	Data processing errors	[15]	10
Insufficient Control Flow Management (CWE-691)	Concurrency	[65]	1
	Incorrect control flow	[65]	1
	Race conditions	[15,42,67]	15
C++ Catastrophes	C++-specific vulnerabilities	[42]	33
Improper Adherence to Coding Standards (CWE-710)	Null pointer dereference	[65]	1
	Double free	[65]	1
	Potentially dangerous function	[42]	2
Improper Check or Handling of Exceptional Conditions (CWE-703)	Improper check or handling of exceptional conditions	[15,65]	22
Incorrect Calculation (CWE-682)	Incorrect calculation	[15,65]	17
	Integer overflows	[42,65]	3
CWE-634 (Deprecated)	Weaknesses affecting system processes (IPC)	[42]	1

Table 7

Primary studies included in this SLR.

Sl. No.	Title	Authors	Year
S1	An Empirical Study of Static Analysis Tools for Secure Code Review [39]	Wachiraphan Charoenwet, Patanamon Thongtanunam, Van-Thuan Pham, Christoph Treude	2024
S2	Toward effective secure code reviews: an empirical study of security-related coding weaknesses [48]	Wachiraphan Charoenwet, Patanamon Thongtanunam, Van-Thuan Pham, Christoph Treude	2024
S3	Empirical analysis of security vulnerabilities in Python packages [66]	Mahmoud Alfadel, Diego Elias Costa, Emad Shihab	2023
S4	Security Defect Detection via Code Review: A Study of the OpenStack and Qt Communities [49]	Jiaxin Yu, Liming Fu, Peng Liang, Amjed Tahir, Mojtaba Shahin	2023
S5	Empirical analysis of security-related code reviews in npm packages [40]	Mahmoud Alfadel, Nicholas Alexandre Nagy, Diego Elias Costa, Rabe Abdalkareem, Emad Shihab	2023
S6	Less is More: Supporting Developers in Vulnerability Detection during Code Review [43]	Larissa Braz, Christian Aeberhard, Gül Çalikli, Alberto Bacchelli	2022
S7	Software Security during Modern Code Review: The Developer's Perspective [41]	Larissa Braz, Alberto Bacchelli	2022
S8	Which bugs are missed in code reviews: An empirical study on SmartSHARK dataset [67]	Fatemeh Khoshnoud, Ali Rezaei Nasab, Zahra Toudeji, Ashkan Sami	2022
S9	Why Security Defects Go Unnoticed during Code Reviews? A Case-Control Study of the Chromium OS Project [15]	Rajshakhar Paul, Asif Kamal Turzo, Amiangshu Bosu	2021
S10	A systematic literature review and taxonomy of modern code review [47]	Nicole Davila, Ingrid Nunes	2021
S11	An Empirical Investigation of Relevant Changes and Automation Needs in Modern Code Review [68]	Sebastiano Panichella, Nik Zaugg	2020

(continued on next page)

Table 7 (continued).

Sl. No.	Title	Authors	Year
S12	Why Did This Reviewed Code Crash? An Empirical Study of Mozilla Firefox [65]	Le An, Foutse Khomh, Shane McIntosh, Marco Castelluccio	2018
S13	A Large-Scale Study of Modern Code Review and Security in Open Source Projects [12]	Christopher Thompson, David Wagner	2017
S14	A security perspective on code review: The case of Chromium [42]	Marco di Biase, Magiel Bruntink, Alberto Bacchelli	2016
S15	Identifying the characteristics of vulnerable code changes: an empirical study [22]	Amiangshu Bosu, Jeffrey C. Carver, Munawar Hafiz, Patrick Hilley, Derek Janni	2014
S16	An Empirical Investigation of Socio-technical Code Review Metrics and Security Vulnerabilities [10]	Andrew Meneely, Alberto C. Rodriguez Tejada, Brian Spates, Shannon Trudeau, DanielleNeuberger, Katherine Whitlock, Christopher Ketant, Kayla Davis	2014
S17	The Impact of Code Review Coverage and Code Review Participation on Software Quality: a case study of the qt, VTK, and ITK projects [17]	Shane McIntosh, Yasutaka Kamei, Bram Adams, Ahmed E. Hassan	2014
S18	Expectations, outcomes, and challenges of modern code review [29]	Alberto Bacchelli, Christian Bird	2013
S19	An Empirical Study on the Effectiveness of Security Code Review [23]	Anne Edmundson, Brian Holtkamp, Emanuel Rivera, Matthew Finifter, Adrian Mettler, David Wagner	2013
S20	Static security analysis based on input-related software faults [64]	Csaba Nagy, Spiros Mancoridis	2009

References

- [1] S. Srinivasan, Q. Pitcher, J.S. Goldberg, Data breach at equifax, in: Harvard Business School Case 118-031, Revised April 2019, Harvard Business School, 2017, URL <https://www.hbs.edu/faculty/Pages/item.aspx?num=53509>.
- [2] D.A. Wheeler, Preventing heartbleed, *Computer* 47 (8) (2014) 80–83, <http://dx.doi.org/10.1109/MC.2014.217>.
- [3] National institute of standards and technology, National vulnerability database, 2024, URL https://nvd.nist.gov/vuln/search/statistics?form_type=Basic&results_type=statistics&search_type=all&isCpeNameSearch=false#.
- [4] A. Ashish, J. Aghav, Automated techniques and tools for program analysis: Survey, in: 2013 4th International Conference on Computing, Communications and Networking Technologies, ICCCNT 2013, 2013, pp. 1–7, <http://dx.doi.org/10.1109/ICCCNT.2013.6726693>.
- [5] B. Johnson, Y. Song, E. Murphy-Hill, R. Bowdidge, Why don't software developers use static analysis tools to find bugs? in: Proceedings of the 2013 International Conference on Software Engineering, ICSE '13, IEEE Press, 2013, pp. 672–681.
- [6] C. Artho, A. Biere, Combined static and dynamic analysis, *Electron. Notes Theor. Comput. Sci.* 131 (2005) 3–14, <http://dx.doi.org/10.1016/j.entcs.2005.01.018>.
- [7] A.F. Ackerman, P.J. Fowler, R.G. Ebenau, Software inspections and the industrial production of software, in: Elsevier North-Holland, Inc., 1984, pp. 13–40, URL <https://api.semanticscholar.org/CorpusID:59830250>.
- [8] A. Ackerman, L. Buchwald, F. Lewski, Software inspections: an effective verification process, *IEEE Softw.* 6 (3) (1989) 31–36, <http://dx.doi.org/10.1109/52.28121>.
- [9] G. McGraw, *Software Security: Building Security In*, Addison-Wesley Professional, 2006.
- [10] A. Meneely, A. Tejada, B. Spates, S. Trudeau, D. Neuberger, K. Whitlock, C. Ketant, K. Davis, An empirical investigation of socio-technical code review metrics and security vulnerabilities, in: 6th International Workshop on Social Software Engineering, SSE 2014 - Proceedings, 2014, pp. 37–44, <http://dx.doi.org/10.1145/2661685.2661687>.
- [11] Y. Shin, A. Meneely, L. Williams, J. Osborne, Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities, *IEEE Trans. Softw. Eng.* 37 (2011) 772–787, <http://dx.doi.org/10.1109/TSE.2010.81>.
- [12] C. Thompson, D. Wagner, A large-scale study of modern code review and security in open source projects, in: Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering, in: PROMISE, Association for Computing Machinery, New York, NY, USA, 2017, pp. 83–92, <http://dx.doi.org/10.1145/3127005.3127014>.
- [13] B. Chess, J. West, *Secure Programming with Static Analysis*, first ed., Addison-Wesley Professional, 2007.
- [14] P.C. Rigby, C. Bird, Convergent contemporary software peer review practices, in: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering, in: ESEC/FSE 2013, Association for Computing Machinery, New York, NY, USA, 2013, pp. 202–212, <http://dx.doi.org/10.1145/2491411.2491444>.
- [15] R. Paul, A.K. Turzo, A. Bosu, Why security defects go unnoticed during code reviews? A case-control study of the chromium OS project, in: Proceedings of the 43rd International Conference on Software Engineering, ICSE '21, IEEE Press, 2021, pp. 1373–1385, <http://dx.doi.org/10.1109/ICSE43902.2021.00124>.
- [16] A. Aurum, H. Petersson, C. Wohlin, State-of-the-art: Software inspections after 25 years, *Softw. Test. Verif. Reliab.* 12 (2002) 133–154, <http://dx.doi.org/10.1002/stvr.243>.
- [17] S. McIntosh, Y. Kamei, B. Adams, A.E. Hassan, The impact of code review coverage and code review participation on software quality: a case study of the qt, VTK, and ITK projects, in: Proceedings of the 11th Working Conference on Mining Software Repositories, in: MSR 2014, Association for Computing Machinery, New York, NY, USA, 2014, pp. 192–201, <http://dx.doi.org/10.1145/2597073.2597076>.
- [18] A. Bosu, J.C. Carver, C. Bird, J. Orbeck, C. Chockley, Process aspects and social dynamics of contemporary code review: Insights from open source development and industrial practice at microsoft, *IEEE Trans. Softw. Eng.* 43 (1) (2017) 56–75, <http://dx.doi.org/10.1109/TSE.2016.2576451>.
- [19] F. Camilo, A. Meneely, M. Nagappan, Do bugs foreshadow vulnerabilities? A study of the chromium project, in: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, 2015, pp. 269–279, <http://dx.doi.org/10.1109/MSR.2015.32>.
- [20] S. Zaman, B. Adams, A.E. Hassan, Security versus performance bugs: a case study on Firefox, in: Proceedings of the 8th Working Conference on Mining Software Repositories, MSR '11, Association for Computing Machinery, New York, NY, USA, 2011, pp. 93–102, <http://dx.doi.org/10.1145/1985441.1985457>.
- [21] A. Meneely, H. Srinivasan, A. Musa, A.R. Tejada, M. Mokary, B. Spates, When a patch goes bad: Exploring the properties of vulnerability-contributing commits, in: 2013 ACM / IEEE International Symposium on Empirical Software Engineering and Measurement, 2013, pp. 65–74, <http://dx.doi.org/10.1109/ESEM.2013.19>.
- [22] A. Bosu, J.C. Carver, M. Hafiz, P. Hilley, D. Janni, Identifying the characteristics of vulnerable code changes: an empirical study, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, in: FSE 2014, Association for Computing Machinery, New York, NY, USA, 2014, pp. 257–268, <http://dx.doi.org/10.1145/2635868.2635880>.
- [23] A. Edmundson, B. Holtkamp, E. Rivera, M. Finifter, A. Mettler, D. Wagner, An empirical study on the effectiveness of security code review, in: J. Jürjens, B. Livshits, R. Scandariato (Eds.), *Engineering Secure Software and Systems*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 197–212.
- [24] N. Munaiah, B.S. Meyers, C.O. Alm, A. Meneely, P.K. Murukannaiah, E. Prud'hommeaux, J. Wolff, Y. Yu, Natural language insights from code reviews that missed a vulnerability, in: E. Bodden, M. Payer, E. Athanasopoulos (Eds.), *Engineering Secure Software and Systems*, Springer International Publishing, Cham, 2017, pp. 70–86.
- [25] O. Foundation, About OWASP, 2013, online, URL https://www.owasp.org/index.php/About_OWASP (Accessed 04 November 2013).
- [26] R. Telang, S. Wattal, An empirical analysis of the impact of software vulnerability announcements on firm stock price, *IEEE Trans. Softw. Eng.* 33 (8) (2007) 544–557, <http://dx.doi.org/10.1109/TSE.2007.70712>.
- [27] B. Boehm, V.R. Basili, Software defect reduction top 10 list, *Computer* 34 (1) (2001) 135–137, <http://dx.doi.org/10.1109/2.962984>.
- [28] MITRE Corporation, Common weakness enumeration, 2024, URL <https://cwe.mitre.org/> [Online]; (Accessed 13 April 2025).
- [29] A. Bacchelli, C. Bird, Expectations, outcomes, and challenges of modern code review, in: 2013 35th International Conference on Software Engineering, ICSE, 2013, pp. 712–721, <http://dx.doi.org/10.1109/ICSE.2013.6606617>.

- [30] M.E. Fagan, Design and code inspections to reduce errors in program development, *IBM Syst. J.* 15 (3) (1976) 182–211, <http://dx.doi.org/10.1147/sj.153.0182>.
- [31] J. Cohen, Modern code review, in: *Making Software: What Really Works, and Why We Believe It*, 2010, pp. 329–336.
- [32] M.V. Mantyla, C. Lassenius, What types of defects are really discovered in code reviews? *IEEE Trans. Softw. Eng.* 35 (3) (2009) 430–448, <http://dx.doi.org/10.1109/TSE.2008.71>.
- [33] M. Beller, A. Bacchelli, A. Zaidman, E. Juergens, Modern code reviews in open-source projects: which problems do they fix? in: *Proceedings of the 11th Working Conference on Mining Software Repositories*, in: *MSR 2014*, Association for Computing Machinery, New York, NY, USA, 2014, pp. 202–211, <http://dx.doi.org/10.1145/2597073.2597082>.
- [34] M. Howard, A process for performing security code reviews, *Secur. Priv. IEEE* 4 (2006) 74–79, <http://dx.doi.org/10.1109/MSP.2006.84>.
- [35] P. Thongtanunam, S. McIntosh, A.E. Hassan, H. Iida, Investigating code review practices in defective files: An empirical study of the qt system, 2015, <http://dx.doi.org/10.1109/MSR.2015.23>.
- [36] B. Kitchenham, S. Charters, et al., *Guidelines for performing systematic literature reviews in software engineering*, 2007.
- [37] G. McGraw, Software security, *IEEE Secur. Priv.* 2 (2) (2004) 80–83, <http://dx.doi.org/10.1109/MSECP.2004.1281254>.
- [38] G. McGraw, Automated code review tools for security, *Computer* 41 (12) (2008) 108–111.
- [39] W. Charoenwet, P. Thongtanunam, V.-T. Pham, C. Treude, An empirical study of static analysis tools for secure code review, 2024, [arXiv:2407.12241](https://arxiv.org/abs/2407.12241), URL <https://arxiv.org/abs/2407.12241>.
- [40] M. Alfadel, N.A. Nagy, D.E. Costa, R. Abdalkareem, E. Shihab, Empirical analysis of security-related code reviews in npm packages, *J. Syst. Softw.* 203 (C) (2023) <http://dx.doi.org/10.1016/j.jss.2023.111752>.
- [41] L. Braz, A. Bacchelli, Software security during modern code review: the developer's perspective, in: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in: *ESEC/FSE 2022*, Association for Computing Machinery, New York, NY, USA, 2022, pp. 810–821, <http://dx.doi.org/10.1145/3540250.3549135>.
- [42] M. di Biase, M. Bruntink, A. Bacchelli, A security perspective on code review: The case of chromium, in: *2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation, SCAM*, 2016, pp. 21–30, <http://dx.doi.org/10.1109/SCAM.2016.30>.
- [43] L. Braz, C. Aeberhard, G. Çalikli, A. Bacchelli, Less is more: supporting developers in vulnerability detection during code review, in: *Proceedings of the 44th International Conference on Software Engineering, ICSE '22*, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1317–1329, <http://dx.doi.org/10.1145/3510003.3511560>.
- [44] M. Kuhrmann, D.M. Fernández, M. Daneva, On the pragmatic design of literature studies in software engineering: an experience-based guideline, *Empir. Softw. Eng.* 22 (2016) 2852–2891, <http://dx.doi.org/10.1007/s10664-016-9492-y>.
- [45] MITRE Corporation, CWE-1000: Research concepts (version 4.20), 2026, <https://cwe.mitre.org/data/definitions/1000.html>. (Accessed May 2026).
- [46] O. Kononenko, O. Baysal, L. Guerrouj, Y. Cao, M.W. Godfrey, Investigating code review quality: Do people and participation matter? in: *2015 IEEE International Conference on Software Maintenance and Evolution, ICSME*, 2015, pp. 111–120, <http://dx.doi.org/10.1109/ICSM.2015.7332457>.
- [47] N. Davila, I. Nunes, A systematic literature review and taxonomy of modern code review, *J. Syst. Softw.* 177 (2021) 110951, URL <https://api.semanticscholar.org/CorpusID:232240592>.
- [48] W. Charoenwet, P. Thongtanunam, V.-T. Pham, C. Treude, Toward effective secure code reviews: an empirical study of security-related coding weaknesses, *Empir. Softw. Eng.* 29 (4) (2024) <http://dx.doi.org/10.1007/s10664-024-10496-y>.
- [49] J. Yu, L. Fu, P. Liang, A. Tahir, M. Shahin, Security defect detection via code review: A study of the OpenStack and qt communities, in: *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM*, IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 1–12, <http://dx.doi.org/10.1109/ESEM56168.2023.10304852>, URL <https://doi.ieeeecomputersociety.org/10.1109/ESEM56168.2023.10304852>.
- [50] J. Davis, Vulnregex-detector: Detect vulnerable regexes in your project, 2021, URL <https://github.com/davisjam/vuln-regex-detector>. (Accessed 14 November 2024).
- [51] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D.M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, Curran Associates Inc., Red Hook, NY, USA, 2020.
- [52] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C.L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback, in: *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, Curran Associates Inc., Red Hook, NY, USA, 2022.
- [53] C.S. Xia, Y. Wei, L. Zhang, Automated program repair in the era of large pre-trained language models, in: *Proceedings of the 45th International Conference on Software Engineering, ICSE '23*, IEEE Press, 2023, pp. 1482–1494, <http://dx.doi.org/10.1109/ICSE48619.2023.00129>.
- [54] H. Li, Y. Hao, Y. Zhai, Z. Qian, Enhancing static analysis for practical bug detection: An LLM-integrated approach, *Proc. ACM Program. Lang.* 8 (2024) 474–499, <http://dx.doi.org/10.1145/3649828>.
- [55] B. Steenhoeck, M.M. Rahman, M.K. Roy, M.S. Alam, E.T. Barr, W. Le, A comprehensive study of the capabilities of large language models for vulnerability detection, 2024, [arXiv:2403.17218](https://arxiv.org/abs/2403.17218), URL <https://doi.org/10.48550/arXiv.2403.17218>.
- [56] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, M. Naik, Understanding the effectiveness of large language models in detecting security vulnerabilities, 2024, [arXiv:2311.16169](https://arxiv.org/abs/2311.16169), URL <https://arxiv.org/abs/2311.16169>.
- [57] Z. Xu, S. Jain, M. Kankanhalli, Hallucination is inevitable: An innate limitation of large language models, 2025, [arXiv:2401.11817](https://arxiv.org/abs/2401.11817), URL <https://arxiv.org/abs/2401.11817>.
- [58] H. Xu, K. Yu, H. He, H. Fang, B. Vasilescu, P.S. Park, The structure of cross-national collaboration in open-source software development, in: *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, 2025, <http://dx.doi.org/10.1145/3746252.3761237>.
- [59] M. Guizani, A. Chatterjee, B. Trinkenreich, M.E. May, G.J. Noa-Guevara, L.J. Russell, G.G.C. Zambrano, D. Izquierdo-Cortazar, I. Steinmacher, M. Gerosa, A. Sarma, The long road ahead: Ongoing challenges in contributing to large OSS organizations and what to do, *Proc. ACM Hum.-Comput. Interact.* 5 (2021) 1–30, <http://dx.doi.org/10.1145/3479551>.
- [60] M. Palazzi, J. Cabot, J.L.C. Izquierdo, A. Solé-Ribalta, J. Borge-Holthoefer, Online division of labour: emergent structures in Open Source Software, *Sci. Rep.* 9 (2019) <http://dx.doi.org/10.1038/s41598-019-50463-y>.
- [61] B. Trinkenreich, M. Guizani, I. Wiese, A. Sarma, I. Steinmacher, Hidden figures: Roles and pathways of successful OSS contributors, *Proc. ACM Hum.-Comput. Interact.* 4 (2020) 1–22, <http://dx.doi.org/10.1145/3415251>.
- [62] J. Linäker, B. Lundell, F. Servant, J. Gamalielsson, S. Muto, G. Robles, Public sector open source software projects - How is development organized? *Empir. Softw. Eng.* 30 (2025) <http://dx.doi.org/10.1007/s10664-025-10626-0>.
- [63] C. Osborne, F. Daneshyan, R. He, H. Ye, Y. Zhang, M. Zhou, Characterising open source co-opetition in company-hosted open source software projects: The cases of PyTorch, tensorflow, and transformers, *Proc. ACM Hum.-Comput. Interact.* 9 (2024) 1–30, <http://dx.doi.org/10.1145/3710944>.
- [64] C. Nagy, S. Mancoridis, Static security analysis based on input-related software faults, in: *Proceedings of the 2009 European Conference on Software Maintenance and Reengineering, CSMR '09*, IEEE Computer Society, USA, 2009, pp. 37–46, <http://dx.doi.org/10.1109/CSMR.2009.51>.
- [65] L. An, F. Khomh, S. McIntosh, M. Castelluccio, Why did this reviewed code crash? An empirical study of mozilla firefox, in: *2018 25th Asia-Pacific Software Engineering Conference, APSEC*, 2018, pp. 396–405, <http://dx.doi.org/10.1109/APSEC.2018.00054>.
- [66] M. Alfadel, D.E. Costa, E. Shihab, Empirical analysis of security vulnerabilities in Python packages, *Empir. Softw. Eng.* 28 (3) (2023) <http://dx.doi.org/10.1007/s10664-022-10278-4>.
- [67] F. Khoshnoud, A.R. Nasab, Z. Toudeji, A. Sami, Which bugs are missed in code reviews: an empirical study on SmartSHARK dataset, in: *Proceedings of the 19th International Conference on Mining Software Repositories, MSR '22*, Association for Computing Machinery, New York, NY, USA, 2022, pp. 137–141, <http://dx.doi.org/10.1145/3524842.3527997>.
- [68] S. Panichella, N. Zaugg, An empirical investigation of relevant changes and automation needs in modern code review, *Empir. Softw. Eng.* 25 (6) (2020) 4833–4872, <http://dx.doi.org/10.1007/s10664-020-09870-3>.